Assessment Tool 2 – Reverse Engineering: Answers

1. Reverse Engineering a Recommendation System

The platform's architecture follows a typical Lambda-style pipeline. Raw clickstream, purchase, and browsing events are ingested continuously and loaded into Spark as RDDs, partitioned by user ID for parallel processing. Data cleaning, deduplication, and feature extraction (recency, frequency, category affinity) are performed using RDD transformations such as map, filter, and reduceByKey, while joins between user and product datasets use join and cogroup operations. Collaborative filtering or matrix-factorization models are trained in batch mode on historical RDDs, and the resulting item-similarity or latent-factor data is cached for fast lookup. Intelligent agents sit on top of this pipeline: a monitoring agent watches real-time user actions, a decision agent re-ranks recommendations using the cached model output combined with contextual signals (time, device, current session), and a feedback agent updates weights based on click-through or purchase outcomes, enabling the system to adapt dynamically without full retraining.

2. Reverse Engineering a Smart Traffic Management System

Vehicle sensors, cameras, and GPS feeds stream data into the system, which is processed using Spark Streaming with RDDs generated in micro-batches (DStreams). Each RDD partition corresponds to a geographic zone or intersection, allowing parallel computation of vehicle density using map and reduceByKey operations, while filter removes noisy or duplicate sensor readings. A sliding-window transformation aggregates congestion trends over short time intervals to detect patterns. Intelligent agents are organized per intersection: a sensing agent collects and preprocesses local data, a decision agent applies rule-based or reinforcement-learning logic to compute optimal signal timing under strict latency constraints, and a coordination agent communicates with neighboring intersection agents to prevent cascading congestion. The distributed, in-memory nature of RDDs ensures the low-latency processing required for real-time signal adaptation.

3. Reverse Engineering a Fraud Detection Pipeline

Transaction data is ingested as a continuous stream and converted into RDDs partitioned by account or transaction ID for parallel feature computation. PySpark transformations extract features such as transaction velocity, geolocation mismatch, and deviation from spending history using map and reduceByKey, while lookup tables of known fraud patterns are joined in using broadcast variables for efficiency. Anomaly detection likely combines a pre-trained model (e.g., logistic regression or isolation forest) applied via mapPartitions for speed, with rule-based thresholds for known fraud signatures. Because the system must respond within milliseconds, the pipeline keeps data in memory and avoids unnecessary shuffles. Intelligent agents include a scoring agent that evaluates each transaction, an alert agent that triggers notifications or blocks transactions exceeding a risk threshold, and a learning agent that periodically retrains the model using newly confirmed fraud/non-fraud labels.

4. Reverse Engineering a Personalized Learning System

Learner interaction data — quiz scores, time-on-task, video engagement, and error patterns — is collected and represented as RDDs keyed by student ID. RDD transformations (map, groupByKey, reduceByKey) compute performance metrics such as mastery level per topic and engagement trends, while filter isolates struggling or excelling learners for targeted intervention. These features feed an adaptive model that predicts appropriate content difficulty. Intelligent agents drive the adaptive behavior: a monitoring agent tracks real-time performance signals, a recommendation agent selects the next piece of content or difficulty level based on the learner's current mastery state, and a feedback agent generates personalized hints or encouragement. The distributed RDD processing allows the system to handle large numbers of concurrent learners while updating each learner's model with minimal latency.

5. Reverse Engineering a Cloud-Based AI Monitoring System

System and application logs from across the cloud infrastructure are streamed continuously and loaded as RDDs partitioned by service or node, enabling parallel log parsing and metric extraction (CPU, memory, latency, error rate) through map and flatMap operations. RDD lineage provides fault tolerance: if a worker node fails, lost partitions are automatically recomputed from the lineage graph rather than requiring data replication, which is essential for uninterrupted monitoring. Aggregation operations such as reduceByKey and windowed computations detect anomalies by comparing current metrics against historical baselines. Intelligent agents coordinate the response: a monitoring agent continuously evaluates incoming metrics, a scaling agent triggers horizontal or vertical resource scaling when thresholds are breached, and an anomaly-response agent isolates or restarts faulty services, with all agents communicating through a shared state store to keep scaling and anomaly-handling decisions consistent.